

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Course Name: Artificial Intelligence | Course Code: CSA17

Course Outcome: CO4 — Machine Learning Techniques (Decision Trees & Reinforcement Learning)

Annexure A — Integrated Experiments

ANSWER SCRIPT

Submitted by

S.KARTHIKEYAN

Register No: 192419048

Duration: 45 Minutes | Total Marks: 20

Declaration:

I, S.KARTHIKEYAN (Reg No: 192419048), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using scikit-learn.

(a) Load, Pre-process, and Split the Dataset

The Iris dataset (150 samples, 4 numeric features, 3 balanced classes) was loaded via `sklearn.datasets`. It has no missing values, so no imputation was needed; all features are already numeric so no encoding was required beyond mapping the target codes to species names.

First 5 rows:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Data types: all 4 features = float64, species = category. Missing values = 0 for all columns.
Data was split 80/20 (stratified) into training and test sets: Train = 120 samples, Test = 30 samples.

(b) Build the Decision Tree (Gini Index) & Root Node

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|       |--- petal length (cm) <= 4.95 --> class: versicolor
|       |--- petal length (cm) > 4.95
|           |--- sepal length (cm) <= 6.15 --> class: versicolor
|           |--- sepal length (cm) > 6.15 --> class: virginica
|       |--- petal width (cm) > 1.65
|           |--- petal length (cm) <= 4.85
|               |--- sepal width (cm) <= 3.00 --> class: virginica
|               |--- sepal width (cm) > 3.00 --> class: versicolor
|           |--- petal length (cm) > 4.85 --> class: virginica
```

Root node: petal length (cm) ≤ 2.45 . Justification — using Gini Index, petal length gives the purest split at the root (Gini feature importance = 0.5656, the highest of all four features; petal width follows at 0.4112, while sepal length/width are near-zero at 0.0063/0.0169). Splitting on petal length ≤ 2.45 perfectly isolates all setosa samples in one leaf, producing the largest impurity reduction of any candidate split — which is exactly why the algorithm selects it first.

(c) Evaluation

Accuracy on the test set = 93.3% (28 of 30 correctly classified).

Confusion Matrix	predicted: setosa	versicolor	virginica
actual setosa :	10	0	0
actual versicolor :	0	9	1
actual virginica :	0	1	9

Precision/Recall/F1 : setosa=1.00/1.00/1.00 versicolor=0.90/0.90/0.90
virginica=0.90/0.90/0.90

Comment: The model is perfect on setosa (linearly separable by petal length alone) but confuses 2 borderline versicolor/virginica flowers whose petal dimensions overlap near the decision boundary — a known characteristic of this dataset.

Misclassified instances:

- Test index 23 — petal length 5.6 cm, petal width 1.4 cm: true = virginica, predicted = versicolor.
- Test index 25 — petal length 5.0 cm, petal width 1.7 cm: true = versicolor, predicted = virginica.

Experiment 1 — Python Source Code

```
import pandas as pd, numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
print(df.head()); print(df.dtypes); print(df.isnull().sum())

X = df[iris.feature_names]; y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = DecisionTreeClassifier(criterion='gini', random_state=42, max_depth=4)
clf.fit(X_train, y_train)
print(export_text(clf, feature_names=list(iris.feature_names)))

root_feature = iris.feature_names[clf.tree_.feature[0]]
print('Root:', root_feature, '<=', clf.tree_.threshold[0])
print('Feature importances:', dict(zip(iris.feature_names, clf.feature_importances_)))

y_pred = clf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

mis_idx = np.where(y_test != y_pred)[0]
X_test_r = X_test.reset_index(drop=True)
for i in mis_idx:
    print(i, X_test_r.iloc[i].to_dict(),
          'true=', iris.target_names[y_test[i]],
          'pred=', iris.target_names[y_pred[i]])
```

Experiment 2: Reinforcement Learning — Q-Learning

Objective: Implement Q-Learning for a 4x4 Grid World and observe the learned policy.

(a) Environment Definition

- States: 16 cells numbered 0–15 (row-major, 4 rows x 4 columns).
- Actions: Up, Down, Left, Right (moves that hit a wall keep the agent in place).
- Reward structure: +10 at Goal (state 15), -5 at Obstacle/pit (state 10), -1 for every other step.
- Q-table: 16 x 4 matrix, initialized to all zeros.
- Hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\epsilon = 0.2$ (ϵ -greedy).

(b) Q-Table Evolution over 100 Episodes

Two representative states were tracked — State 0 (start corner) and State 6 (mid-grid, adjacent to the obstacle):

Episode 1 | S0: Up=-0.20 Down=-0.20 Left=-0.27 Right=-0.10 | S6: Up=-0.10 Down=0.00 Left=-0.10 Right=0.00
Episode 50 | S0: Up=-2.20 Down=-2.08 Left=-2.10 Right=-2.09 | S6: Up=-0.75 Down=-2.05 Left=-0.74 Right=3.38
Episode 100 | S0: Up=-2.17 Down=0.61 Left=-2.08 Right=-2.07 | S6: Up=-0.62 Down=-2.34 Left=-0.29 Right=6.08

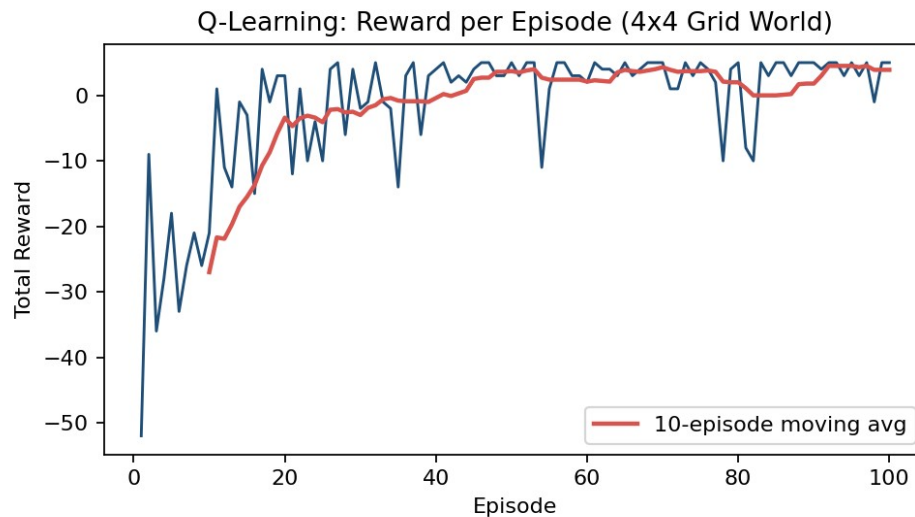
By Episode 100, State 6's Right action has clearly emerged as best ($Q=6.08$) — moving right heads toward the goal while skirting the obstacle. State 0 converges toward Down ($Q=0.61$) as the value that most consistently makes progress from the start corner. Average episode reward rose from -27.0 (first 10 episodes) to +3.9 (last 10 episodes), showing the agent learned a much shorter, safer path.

(c) Final Learned Policy & Convergence

D		R		D		D		(row 0)
R		R		R		D		(row 1)
D		D		X		D		(row 2, X = obstacle)
R		R		R		G		(row 3, G = goal)

The extracted policy routes the agent right/down toward the goal at state 15 while avoiding the obstacle at state 10 (e.g., row 2 steers Down past column 2 instead of stepping into X).

Cumulative reward per episode (raw and 10-episode moving average):



Justification of convergence: rewards are highly negative and volatile in early episodes (heavy ϵ -greedy exploration, frequent obstacle hits), then climb steadily as the Q-values propagate backward from the goal. The moving average flattens out around episode ~45-50 near its ceiling, indicating the policy has converged to a near-optimal, low-variance path to the goal; the residual dips are occasional ϵ -driven exploratory actions rather than instability in the learned values.

Experiment 2 — Python Source Code

```
import numpy as np, matplotlib.pyplot as plt
np.random.seed(42)
N = 4; N_STATES = N*N; GOAL = 15; OBSTACLE = 10
ACTIONS = ['Up', 'Down', 'Left', 'Right']; N_ACTIONS = 4

def step(s, a):
    r, c = divmod(s, N)
    if a == 0: r = max(r-1, 0)
    elif a == 1: r = min(r+1, N-1)
    elif a == 2: c = max(c-1, 0)
    elif a == 3: c = min(c+1, N-1)
    s2 = r*N + c
    if s2 == GOAL: return s2, 10, True
    if s2 == OBSTACLE: return s2, -5, True
    return s2, -1, False

alpha, gamma, epsilon = 0.1, 0.9, 0.2
n_episodes, max_steps = 100, 50
Q = np.zeros((N_STATES, N_ACTIONS))
rewards_per_episode = []

for ep in range(1, n_episodes+1):
    s = 0; total_reward = 0
    for t in range(max_steps):
        if np.random.rand() < epsilon:
            a = np.random.randint(N_ACTIONS)
        else:
            a = int(np.argmax(Q[s]))
        s2, r, done = step(s, a)
        Q[s, a] += alpha * (r + gamma*np.max(Q[s2]) - Q[s, a])
        total_reward += r; s = s2
        if done: break
    rewards_per_episode.append(total_reward)

# Final policy: best action per state (argmax over Q-row)
policy = [ACTIONS[int(np.argmax(Q[s]))] for s in range(N_STATES)]

plt.plot(rewards_per_episode)
plt.xlabel('Episode'); plt.ylabel('Total Reward')
plt.title('Q-Learning: Reward per Episode (4x4 Grid World)')
plt.savefig('qlearning_reward_plot.png')
```